



Introduction To Linux Operating System

File System

Prof. Badre BOUSALEM



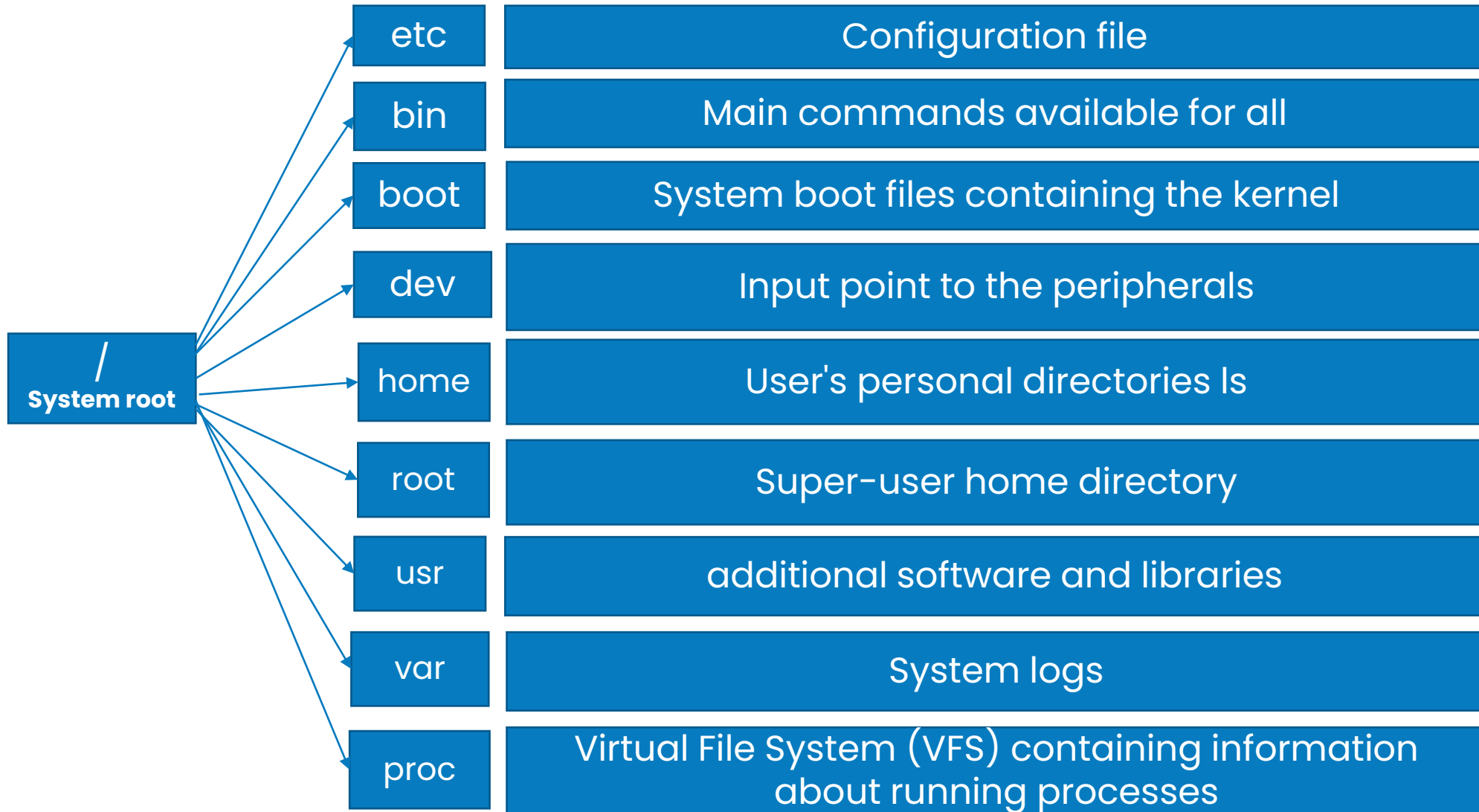
Plan du cours

- I. Linux file system
- II. Main file management commands.
- III. File access permissions
- IV. File property management
- V. Symbolic or physical links
- VI. Management of system files
- VII. Vi/vim editors

I. Linux file system



Tree structure of the Linux file system



Contents of the directories

./	Root directory
bin	Basic command binaries for all users
boot	Boot loader file
dev	Access points prepared by the system to the devices
etc	Configuration files
lib	Shared libraries and kernel modules
media	Assembly points of the removable supports
mnt	Assembly points for installing temporary filesystems
opt	Installation directory for some optional applications
proc	Virtual file system containing kernel and process information
sbin	Essential system binaries
srv	Data about the services provided by the system
Tmp	Temporary files
usr	Subdirectories containing tools & user applications
var	Variable data
home	Home directories for all users

The different categories of files (1/2)

Normal files:

- Text: mail, program sources, scripts, configuration ...
- Executables: programs in binary code

Directory files:

These are container files that contain references to other files.

The different categories of files (2/2)

To list the contents of a directory in detail, type the command `ls -l`

```
lrwxrwxrwx  1 root root      15 2010-02-10 15:46 libz.so.1 -> libz.so.1.2.3.3
-rw-r--r--  1 root root  83608 2009-09-08 12:01 libz.so.1.2.3.3
drwxr-xr-x  2 root root   4096 2009-10-28 21:58 linux-sound-base
```

-	ordinary file
d	directory file
c	special file: device mode character (raw)
b	special file: block mode device
p	named pipe
l	symbolic link
s	socket

Data exchanges between this file and system are done by bytes (example: terminals, serial and parallel ports)

Data exchanges between this file and system are done by blocks (example disk drive)

Unidirectional inter-process communication file

Shortcut to other file

Bidirectional communication file allowing to communicate with another process on the same machine or with a process running on other machines.

Directories, files, inodes

- Each directory/file is listed in its parent directory (except rootDir)
- The directory is a file mapping
- filename or RepName ----->inode numberAn inode is a unique number that references one or more files in the file system. This number is the location element of the file on the file system.
- The inode represents the physical reality of the file to which one or more names (locations on the file system) correspond.
- An inode is a file descriptor: inode number ---->permissions, UID, GID, size, dateAccess etc.
- The inode table contains information about files

Main folders, terms and included utilities

A valid file name:

- Must contain a maximum of 255 characters per filename.
- Is case sensitive: 'A' and 'a' are different.
- Can contain any character except "/".

It is recommended to use alphanumeric characters as well as plus ("+"), minus ("-"), dot ("."), comma (",") and underscore ("_").
Accented characters and spaces should be avoided, but this is not a rule.

Characters to avoid

When naming a file, there are certain characters that should be avoided:

- Hyphens at the beginning of the name.

```
[root@test] /# touch my-file -lt
```

- Spaces
- All other special characters: !@##%^&*() ; , ' " { | \ < > ? ~ `

II. Main file management commands



Perform basic file management

ls: Lists the contents of directories.

file: Finds information about files (type)

whereis: Locate the binary, source and manual page files for a command

which: Locate the command

mv: Moves (renames) files.

cp: Copies files or directories.

touch: Changes the time stamp of the file or creates a new file.

rm: Deletes directories or files.

mkdir: Creates a directory.

rmdir: Deletes a directory.

find: Search for files in a directory tree.

Main file management commands

cat, tac: Link files and print standard output.

cut: Deletes sections from each line of the files.

expand: Converts tabs to spaces.

unexpand: Convert spaces to tabs.

fmt: Formats text.

head, tail: Displays the first or the last part of the file.

join: Merges the lines of two files into a common field.

nl: Displays the line numbers of the file.

od: Transfer files in octal or other formats.

paste: Merges the lines of the files.

pr: Convert text files for printing.

sed: Allows to make text substitutions.

sort: Extracts lines from text files.

split: Split a file into pieces.

tr: Transfer or delete characters.

uniq: Deletes duplicate lines from a matched file.

wc: Shows the number of bytes, words and lines in the file.

File globbing and wildcards (1/2)

- File globbing is the Shell's expansion of file names based on special characters or wildcards. Wildcard patterns produce a set of file names in a single expression.
- The term "globbing" comes from the "glob" sub-program, which handled wildcards in pre-Sbourne versions of Unix shells.
- The purpose of wildcards is to increase productivity:
 - Find files you can't remember.
 - Find files that have something in common.
 - Working with groups of files rather than file by file.
- The shell is able to interpret these special characters:
! @ # % ^ & * () { } [] | \ ; ~ ' " ` ?
- The characters used for wildcards are as follows:
? * [] ~ !

File globbing and wildcards (2/2)

➤ Special wildcard characters:

?	Corresponds to any single character
*	All characters
[abcfghz]	One character among
[a-z]	A character in the rank
[!x-z]	Is not on the rank
~	Home directory
~user	User's home directory

Examples:

?	File names composed of 1 character
[aA]????	Five characters, beginning with a or A.
~toto	Path name of toto's home directory
[!0-9]*	All strings that do not start with a number.

Using wildcards in the Shell

A Shell command line can be simple or more complex.

```
[root@test ] /# ls -l [fF]*  
[root@test ] /# ls *.c | more  
[root@test ] /# ls -l [a-s]* | mail `users`
```

- The first action of a Shell is to expand file names and variables, which allows all wildcards to be interpreted.
- If a wildcard pattern cannot be expanded, the Shell interprets it as if it were a normal file name.
- The Shell interprets all wildcards it finds. To avoid this, you have to use the quoting mechanisms (see previous chapter).

III. File access permissions



Users, groups, permissions

- Users are associated with groups.
 - The system determines whether a user/group can access a file/directory (access permissions).
 - Root: permission to access any file/directory.
- 

Access Permissions

The default permissions are:

- 0666 for creating a file
- 0777 for creating a folder

Extra access rights			user			group			others		
SUID (s)	SGID (s)	Sticky Bit (t)	r	w	x	r	w	x	r	w	x
4	2	1	4	2	1	4	2	1	4	2	1

R→ allows reading of the directory contents

X→ allows access to the directory (cd) and execution of a file

W→ allows creation, deletion and renaming of a directory item (**mkdir, rm, rmdir, mv**)

Basic rights management

user			group			others		
r	w	x	r	w	x	r	w	x
4	2	1	4	2	1	4	2	1

Droits	Octal	Binaire
---	0	000
--x	1	001
-w-	2	010
-wx	3	011
r--	4	100
r-x	5	101
rw-	6	110
rwX	7	111

chmod 621 exple

└───────────> **rw- -w- --x**

chmod 700 exple

└───────────> **rwX --- ---**

chmod 754 exple

└───────────> **rwX r-x r--**

Default permissions

- The default permissions are :
 - 0666 for creating a file
 - 0777 for creating a folder
- Most systems assign these permissions at boot time with the umask command.
- In general, the mask value is 022. This means that write access for the group and other users is restricted.
- To determine the current value of umask: **\$ umask**
- To change the value of umask: **\$ umask 77**

/etc/login.defs

IV. File property management



Manage file property

Change the owner

chown [-R] nv_user files

- Command reserved for the current owner of the files or directories (and for root).
- The option -R (recursive) allows to act on all the subdirectories.

Example :

```
chown -R stage4 /home/stage1
```

Change the owner group

chgrp [-R] nv_groupe files

This must be done by root or the owner, provided that the owner is a member of the new group.

Example :

```
chgrp -R stage4 /home/stage1
```

V. Symbolic or physical links



Create and change symbolic or physical links (1/3)

Links are used to :

Create a new path to a file.

Create a shorter alias to a file. To link one file to another, use ln.

```
ln [options] filename linkname  
ln [options] filename linkdirectory
```

Main options :

f: force: Overwrites existing links

s: Symbolic link.

By default, links are physical links. In Windows, they are called "shortcuts" (raccourcis)

Create and change symbolic or physical links (2/3)

Properties of real (physical) links:

- A physical link can only be created to an existing file.
- A file can only be deleted after having deleted all its physical links.
- The link and its target must be on the same drive.
- Physical links cannot extend across a file system or multiple partitions.
- The physical link references its target by its inode number.
- Even with the correct rights, it is impossible to create a link to a directory.

Properties of symbolic links :

- A symbolic link can be created to a file that does not yet exist and is identified by the `ls` command.
- Thus, it is possible to break a symbolic link by deleting its target.
- The symbolic link references its target with its name and path, not with its inode number.
- Symbolic links can be deployed on multiple file systems or drives.
- The size of a symbolic link depends on the size of the target's name.

Create and change symbolic or physical links (3/3)

- Physical and symbolic links can be displayed with the command `ls -l`.
- Symbolic links are marked with "l" in the first position of the permissions triplet.
- They are also indicated with arrows: `link_name → real_filename`
- To delete a link, you should use the command **rm**.
- Only the link will be deleted and not the file

```
[root@test ] /# ls -l /dev/midi
lrwxrwxrwx 1 root root 6 Jul 4 21:50 /dev/midi -> midi00
```

- Physical links are indicated with the link number.

Example:

```
[root@test ] /# ls ~/readme
lrwxrwxrwx 3 rarrigoni users 677 Jul 4 21:50 readme
```

VI. System file management



Find and place system files

Commands :

Find: search for all files

Locate: allows to find a file very quickly

Whereis: is used to locate the binary, source and man page files of a command.

Which: used to identify the location of a given executable that is executed when you type the name of the executable (command) in the terminal prompt.

Whatis: the command is used to get a one-line description of a man page.

→ To see the function of the different command options type :
man [command]

VII. VI/VIM editor



VI/VIM editor (1/2)

VI is one of the most popular text editors under Linux (along with Emacs and pico) despite its very limited ergonomics.

VI (pronounced Vihaille) is entirely in text mode, which means that every action is done using text commands.

→ **Very powerful, very useful in case of a malfunction of the graphical interface.**

Vim : it is an improved version of the text editor VI

Vim (*VI iMproved*, improved version of Vi) is widely used and generally available by default on most Unix-based OS, such as Linux.

VI et VIM have two modes:

Command mode: Everything you type will be interpreted as a command.

Edit mode: Everything you type will be inserted into a file.

VI/VIM editor(2/2)

To install the VI/VIM tool enter:

```
sudo apt-get install vim
```

```
zouhour@zouhour-virtual-machine:~$ sudo apt-get install vi
```

The command to launch VI or VIM is:

```
vi nom_du_fichier
```

```
~$ vim nom_du-fichier
```

VI editor: basic commands

Command	Description
:q	Exit the editor (without saving)
:q!	force the editor to exit without saving (even if changes have been made to the document)
:wq	Save the document and exit the editor
:file <i>file_name</i>	Save the document as

VI editor: Edition commands

Command	Description
i	Before the cursor
I	At the beginning of the line
a	After the cursor
A	At the end of the line
o	Add a line after the current line
O	Add a line before the current line
x	Delete the current character under the cursor
dd	Delete the current line under the cursor
dxd	Delete x lines from the current one under the cursor
nx	Delete n characters from the current one under the cursors
x>>	Move x lines to the right from the current one under the cursor
x<<	Move x lines to the left from the current one under the cursor

VI editor

How to switch from one mode to another?

From command mode to edit mode: with the keys i, I, a, A, o, O

From edit mode to command mode: ESC key.

The default mode at launch is the command mode

VIM Editor: Learning with Vimtutor

For the new users, Vim integrates a real small tutorial This program can be launched by typing :

```
vimtutor
```

- Vimtutor launches Vim by opening a predefined help file.
- This introduction to Vim is in French and accessible to everyone